

Generating Test Orders for Efficient Order-Dependent Flaky Test Detection

Anonymous Author(s)

Submitted for double-anonymous review

Abstract—Flaky tests that can pass or fail even when run on the same code are a major challenge in software testing. Order-dependent (OD) tests are a prominent type of flaky test that exhibit different test outcomes based on the order in which tests are run. To detect OD tests, prior work proposed running tests in random test-orders, identifying tests that fail in one test-order but pass in another as OD tests. Subsequent research proposed a more systematic approach to generate test-orders that provide stronger guarantees for detecting all OD tests. However, these approaches are expensive, often generating more test-orders and running more tests than needed to satisfy their goals.

We propose TuscanE, a new systematic approach that generates test-orders to cover “test-pairs”, where a test-pair is covered if a test-order contains the two tests positioned right next to each other, with no other tests in-between. TuscanE specifically aims to maximize the number of newly covered test-pairs in each test-order. TuscanE will also run only a subset of all tests in a test-order if it determines that including some tests would only cover test-pairs already covered in prior test-orders.

We evaluate TuscanE on 46 test suites from popular open-source projects, where TuscanE generates test-orders at a 1 : 7.95 : 8.15 ratio compared to two prior approaches while still ensuring complete coverage of all test-pairs. TuscanE also generates all such test-orders within a reasonable time of 3.39 seconds on average. When we consider the total time to generate and run test-orders, TuscanE is faster than all prior approaches (372,967 seconds for TuscanE vs. 9,976,133 seconds and 2,687,613 seconds for the other approaches). Compared to prior approaches, TuscanE runs fewer test-orders and tests, yet still covers all test-pairs and detects all OD tests.

Index Terms—regression testing, flaky test, order-dependent, combinatorial design theory

I. INTRODUCTION

Regression testing helps developers ensure quality and resilience in software as it evolves over time. Unfortunately, regression testing often suffers from *flaky tests* [1], i.e., tests that can pass and fail when run on the same code. Flaky tests are prevalent in open-source projects and industry code [2]–[8], posing a serious problem for developers, with some industry researchers even suggesting that developers should “assume all tests are flaky” [3]. Despite this assumption, developers still strive to detect and fix flaky-test failures, e.g., Google aims to detect flaky behavior before CI [9], [10] and Microsoft invests in debugging and fixing flaky tests [11], [12].

One prominent type of flaky test is *order-dependent (OD) tests* [1], [2], [13], [14], which are tests whose pass and fail outcomes depend on the order in which they are run. An OD test fails when another test “pollutes” or does not “set up” some shared global state that the OD test depends upon. Prior work proposed various approaches to detect OD tests,

including generating random orders [2], [13]–[15], leveraging code changes [16], and analyzing code dependencies between tests [17], [18]. Recent work [19], [20] explored systematically generating a set of *test-orders*, where each generated test-order is a complete permutation of the full test-suite. Compared to techniques that randomly generate test-orders, systematic generation can guarantee the detection of OD tests and determine when generation should stop. These generated test-orders ensure that for every pair of tests $\langle t_1, t_2 \rangle$, there exists a test-order among the set where t_1 is positioned right before t_2 with no other test is in-between the two, and there exists another test-order where t_2 is positioned right before t_1 ; we say that $\langle t_1, t_2 \rangle$ is then *covered* by these test-orders. Running the set of generated test-orders guarantees the detection of all OD tests whose outcomes depend on just one other test, which prior work [21], [22] found can detect almost all OD tests, as each OD test usually depends on just one other test.

Li et al. [20] proposed Tuscan inter-class, which is a technique to generate test-orders that cover all test-pairs. Despite its advantages, Tuscan inter-class is very expensive to use, as it generates many test-orders to be practical (an average of 769,148 test-orders per test suite). The main difficulty in covering all test-pairs efficiently comes from needing to cover *inter-class test-pairs*, where the two tests come from different test classes, because a test-order cannot interleave tests in different test classes with each other [2]. An inter-class test-pair can only be covered in a test-order where the two test classes are positioned next to each other, and only one inter-class test-pair involving the two classes can be covered in a given test-order. Tuscan inter-class addresses this problem in a brute-force manner, where many test-orders may redundantly cover test-pairs already covered by prior test-orders.

While a natural alternative to cover test-pairs would simply construct test-orders consisting of just each test-pair, this pairwise approach also comes with limitations, such as the overhead cost of each pair run. The time to run a sequence of tests involves not just the time to run each individual test in that sequence but also the overhead to set up the environment to run tests, along with the time to clean up the environment when tests finish [23]. A pairwise approach will generate many more test-orders, where each one runs just two tests, but each run incurs a large overhead, culminating in inefficient runtime. An ideal approach will need to run not just a small number of tests, but also a small number of orders.

We propose *TuscanE*, an approach for systematically generating an efficient set of test-orders. TuscanE improves upon

prior work [20] in the following: (1) most generated test-orders maximize the number of not covered inter-class test-pairs (5.45 inter-class test-pairs covered per test-order on average by TuscanE vs. 0.99 for Tuscan inter-class and 0.86 for pairwise), (2) test-orders more likely to reveal OD tests are run first (based on heuristics from prior work [20]), and (3) most tests that do not contribute to covering new test-pairs are skipped. In comparison to the pairwise approach, TuscanE covers many more test-pairs per test-order by running more than two tests, which covers at most one new test-pair. By covering more test-pairs per test-order, TuscanE runs fewer test-orders, incurs lower overhead costs, and have a lower total runtime.

We evaluate TuscanE by generating test-orders for 46 test suites of open-source projects, found in prior studies [20] to contain OD tests. We compare the number of generated test-orders against Tuscan inter-class [20] and a pairwise approach [13], [24]. Our results show that TuscanE generates much fewer test-orders compared against Tuscan inter-class and pairwise, at a 1 : 7.95 : 8.15 ratio compared to the number of test-orders that Tuscan inter-class and pairwise generate, respectively. TuscanE generates all such test-orders in just 3.39 seconds on average, compared to 925.20 seconds taken by Tuscan inter-class and 0.76 seconds by Pairwise. Overall, TuscanE has the lowest total time to generate and run all test-orders to cover all test-pairs and detect all OD tests, taking 372,967 seconds compared with 9,976,133 seconds for Tuscan inter-class and 2,687,613 seconds for Pairwise.

Beyond detecting all OD tests, developers may also value the efficient detection of the first OD test. TuscanE detects the first OD test earlier than Tuscan inter-class and Pairwise, requiring only 6.67 test-orders and 196.06 seconds on average, compared with test-orders and seconds for Tuscan inter-class and test-orders and seconds for Pairwise. Compared against randomly generating test-orders, TuscanE requires an initial upfront cost in generating test-orders before running, while random generation allows running test-orders immediately, potentially detecting OD tests earlier, in a limited time budget. However, even in this setting, we find TuscanE out-performing random generation, detecting the first OD test with only 6.67 test-orders and 196.06 seconds on average, compared with 8.22 test-orders and 243.94 seconds for random generation.

This paper makes the following main contributions:

Systematically generating test-orders. We propose a new approach, TuscanE, that generates a more efficient set of test-orders compared to prior approaches [13], [20], [24] while still covering all test-pairs. Running all generated test-orders provides the same guarantee of detecting all common OD tests. **Evaluation.** We evaluate TuscanE on 46 test suites of open-source projects that contain OD tests found in Tuscan inter-class [20]. TuscanE generates much fewer test-orders at a 1 : 7.95 : 8.15 ratio compared to Tuscan inter-class and Pairwise, while taking only 3.39 seconds to generate. We also evaluate first-OD test detection and find that TuscanE detects the first OD test earlier than Random, Tuscan inter-class, and Pairwise, requiring only 6.67 test-orders and 196.06 seconds on average. We make our approaches and data publicly available [25].

II. BACKGROUND

OD test definitions. OD tests are flaky tests that pass or fail based on the order in which tests are run [1], [2], [13], [26]. An OD test always passes or always fails when run in a given test-order. By definition, each OD test must have at least one test-order in which the test passes and at least one test-order in which the test fails. Shi et al. [21] categorized OD tests into two categories, victims and brittles. *Victims* are tests that pass when they are run on their own, but fail when another test, called a *polluter*, runs before the victim. The reason for why a victim fails when a polluter runs first is because the polluter “pollutes” some state that is used by the victim. Aside from polluters, there are also *cleaners*, which are tests that “clean” the state polluted by polluter. Cleaners complicate the detection of OD tests as running a single test-order forward and backward may not result in a victim failing (e.g., running $\langle \text{Polluter}, \text{Cleaner}, \text{Victim} \rangle$ and $\langle \text{Victim}, \text{Cleaner}, \text{Polluter} \rangle$ has the victim pass in both test-orders). Although each victim may have many different polluters and each polluter may have many different cleaners, prior work [21] found that a polluter usually consists of just one test (i.e., a victim can fail due to just one test running before) and recommended focusing on running individual tests before OD tests as an effective way to detect them. The other type of OD test, *brittles*, fail when the test is run by itself, but passes when it is run after another test called a *state setter*. In other words, a brittle requires the state setter to set up the necessary state shared between the tests for the brittle to pass.

OD test example. Figures 1a and 1b show an example of a victim OD test and polluter, respectively, from our evaluation subjects. The victim, `testGetInvoker`, aims to check whether the `invoker` (Line 4) is properly invoked and returns the expected “aa” value (Line 5). The test sometimes fails because test `testAsync`, the polluter, can write a new object using the shared `RpcContext` object (Line 15 of Figure 1b) used also by the `invoker` in the victim. If `testAsync` is run before `testGetInvoker`, then the assertion on Line 5 fails as the object from Line 15 is returned instead of the expected value. If `testGetInvoker` is run before `testAsync`, then both tests pass. In the test suite, `testGetInvoker` has one polluter (`testAsync`) and four cleaners. Of the four cleaners, one is in the same test class as the polluter (`RpcContextTest`), while the other three are all in different test classes as the polluter and victim.

OD test detection work. Prior approaches to detect OD tests, including generating random test-orders [2], [13]–[15], using code changes [16], and analyzing the code [17], [18], provide no guarantees that OD tests will be detected, as random test-orders may not place the polluter or state-setter before the OD test to expose flakiness, therefore requiring an unreasonable number of orders to detect all OD test (up to $N!$, where N is the number of tests). Prior work [13], [24] also proposed *Pairwise*, i.e., running two test-orders for every pair of tests – once as (T_1, T_2) , then as (T_2, T_1) . Recent work [19], [20] introduced the use of *Tuscan squares* [28]

```

1 static ProxyFactory fact = new JdkProxyFactory();
2 @Test public void testGetInvoker() ... {
3     DemoService origin = new DemoServiceImpl();
4     Invoker<DemoService> invoker = fact.getInvoker(...);
5     Assert.assertEquals(invoker.invoke(...).getValue(),
6         origin.echo("aa"));
7 }

```

(a) Victim OD test from JdkProxyFactoryTest class

```

10 @Test public void testAsync() {
11     AsyncContext asyncContext = new AsyncContextImpl(...);
12     RpcContext rpcContext = RpcContext.getContext();
13     rpcContext.setAsyncContext(asyncContext);
14     rpcContext.startAsync();
15     asyncContext.write(new Object());
16     ... }

```

(b) Polluter from RpcContextTest class

Fig. 1: Example victim and polluter in the dubbo-rpc-api project [27]

to systematically generate test-orders to run and detect OD tests that require just one other test to fail. Given a test suite with N tests, Tuscan squares will generate N permutations, where each permutation will have all N tests in different orderings. The set of N permutations generated by Tuscan square guarantees that for any pair of tests, $\langle t_1, t_2 \rangle$, there exists a permutation where t_1 is right before t_2 (with no other test is in-between them), and there exists another permutation where t_2 is right before t_1 . Tuscan squares is guaranteed to need only N permutations for any value of N , except $N = 3$ or $N = 5$, which requires four and six test-orders, respectively. For example, to ensure any pair of test is run right next to each other for a test suite of three tests ($N = 3$), Tuscan squares requires four test-orders to cover all six test-pairs: $\{\langle t_1, t_2, t_3 \rangle, \langle t_2, t_1, t_3 \rangle, \langle t_3, t_2, t_1 \rangle, \langle t_3, t_1, t_2 \rangle\}$.

Li et al. [20] extended the work of Wei et al. [19] for Maven-based, Java projects by implementing *Tuscan inter-class*, which creates each test-order by using Tuscan squares to first create a test class order and then Tuscan squares again to create a test order within each test class. When tests are executed using JUnit [29] and Surefire [30] (the primary unit-testing infrastructure for Maven-based, Java projects), *tests from different test classes cannot be interleaved with each other and each test may be executed at most once in a test-order*. In other words, each test-order cannot run a test from one class, e.g., class A, then a test from another class, then another test from class A. Each order also cannot run the same test multiple times. Violating either of these requirements may provide false positives (i.e., unexpected test failures). These false positives mean that tests will fail not because they are genuinely OD tests that will fail when run in different test-orders, but because they are run in a way that is not usually possible with developer's test infrastructure. Depending on whether t_1 , t_2 , and t_3 from our prior example are in the same test class or not, Tuscan squares may need more than four test-orders to cover all pairs. For example, if t_1 is in class C_1 and t_2 and t_3 are in C_2 , then $\langle C_2.t_2, C_1.t_1, C_2.t_3 \rangle$, $\langle C_2.t_3, C_1.t_1, C_2.t_2 \rangle$ are not valid test-orders. In this example, $\langle C_2.t_2, C_2.t_3 \rangle$ is an *intra-class pair* (test pairs from the same test class), while $\langle C_1.t_1, C_2.t_2 \rangle$ is an *inter-class pair* (test pairs from different test classes). If all three tests are in the same test class, four unique test pairs can be covered with just two test-orders ($\langle t_2, t_1, t_3 \rangle, \langle t_3, t_1, t_2 \rangle$). However, if t_1 is in a different test class, covering just the same four pairs would already need *four* test-orders. This restriction on interleaving also applies to Java test classes from different Maven modules, Python tests from different test files, and so on.

Although, both of Pairwise and Tuscan inter-class can help detect all possible OD tests that depend on just one other test, which prior work [21], [22] found to be the majority of OD tests, we find that they run many unnecessary test-orders and tests, thereby substantially increasing their cost to detect all OD tests. These prior approaches and TuscanE takes a set of fully-qualified test names as input to output a set of orders, where each element of the set is a permutation of test names. When all orders from the output set are run, any pair of tests in any permutation will be guaranteed to have been run consecutively in one or more test-orders.

III. APPROACHES TO GENERATE TEST-ORDERS TO DETECT ALL OD TESTS

The runtime to detect all possible OD tests that depend on just one other test is determined by: (1) number of test-orders and (2) number of tests to run so that for any two tests, T_1 and T_2 , there exists at least one order in which T_1 is immediately before T_2 and at least one order in which T_2 is immediately before T_1 . Each test-order encompasses not only the time of each test, but also the time to setup and teardown test dependencies (e.g., creating test related files). As our experiments show later, this setup and teardown overhead time can be nontrivial (Table I). An efficient approach at generating test-orders to detect all OD tests will ideally run a small number of test-orders and a small number of tests.

A. Generating Test-Orders with TuscanE

We propose *TuscanE*, a new approach to generate test-orders systematically in a way that ensures 100% coverage of test-pairs, while minimizing the number of generated test-orders. We take a two-step approach for generating test-orders, where the first step focuses on covering intra-class and class pairs, while the second step focuses on covering inter-class test-pairs. Covering class pairs means that each pair of test classes has at least one order where the classes are before each other and another where the classes are after each other. We cover these test-pairs first because Li et al. [20] found that covering these test-pairs already detects most OD tests. Once we cover all intra-class and class pairs, we do not need to run all tests within a given class, which enables TuscanE to focus on maximizing the coverage of inter-class test-pairs without running extraneous tests per order in the second step.

The left side of Figure 2 shows how TuscanE generates test-orders for an example test suite containing three test classes: A, B, and C, which have two, four, and two tests, respectively. The first four orders that TuscanE generates will cover all

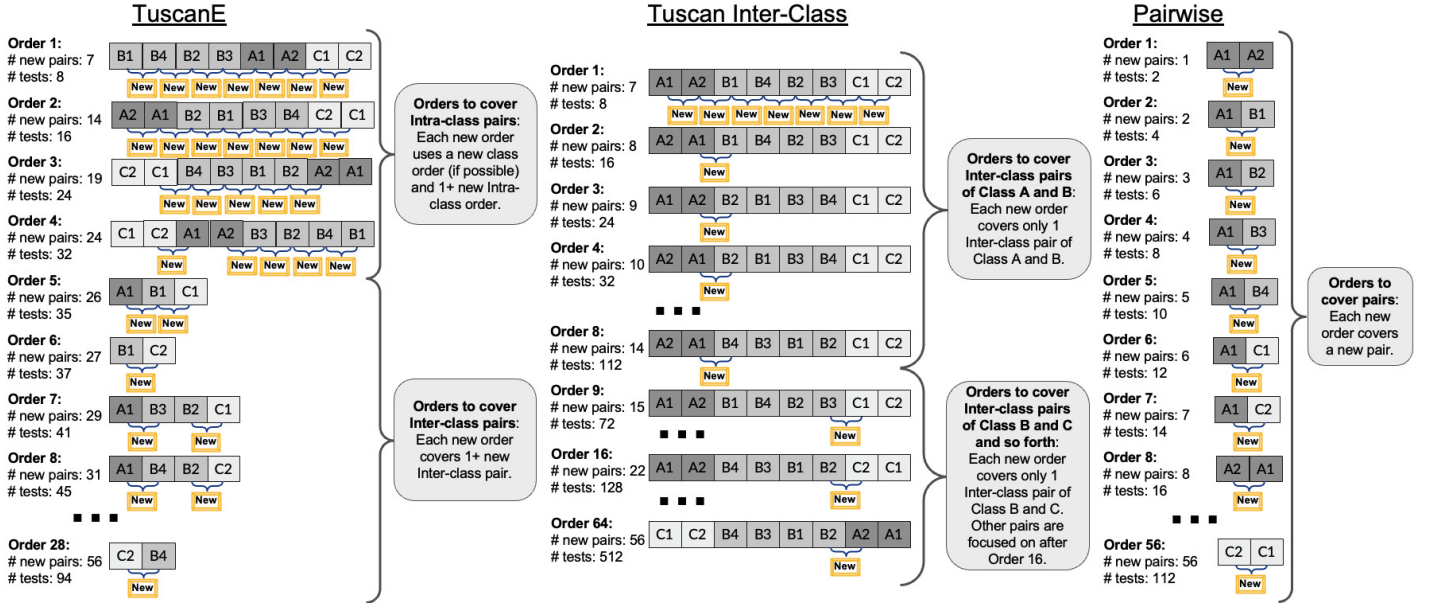


Fig. 2: Example of our proposed approach (TuscanE), Tuscan inter-class approach [20], and a pairwise approach [13], [24]. “# new pairs” and “# tests” is the cumulative number of new pairs covered and tests run (respectively) after the order is run.

intra-class and class pairs. TuscanE first orders the classes using Tuscan Squares and outputs Order 1 as (B→A→C). TuscanE then uses Tuscan Squares to generate an order for the tests inside each class (e.g., A1→A2 for class A). These first few test-orders cover all intra-class and class pairs, while covering some inter-class test-pairs.

Once all intra-class and class pairs are covered, TuscanE simply focuses on covering as many inter-class pairs as possible in each new test-order. Starting at Order 5 of Figure 2, TuscanE generates test-orders focused on covering the inter-class test-pairs. Note that, with any test-order, we can cover at most $N - 1$ number of inter-class test-pairs, one per each pair of test classes adjacent to each other in the test-order (where N is the number of classes). For each pair of adjacent test classes, an inter-class test-pair forms from the last test of the first test class and the first test of the second test class.

TuscanE covers inter-class pairs by first creating a Tuscan square for the test classes. For each class row in the Tuscan square, e.g., A→B→C, TuscanE computes all possible combinations of tests between each adjacent pair of test classes (A→B and B→C). For example, if TuscanE wants to cover the inter-class test-pair (A1, B1), TuscanE will generate a test-order where test A1 is positioned as the last test within its test class A, and the first test within class B is B1. If TuscanE needs to also cover inter-class test-pair (B1, C1), it will ensure that the last test of class B is B1, while the first test of class C is C1. In this example, the only way for a single test-order to cover both test-pairs is to have B1 be the only test in class B, allowing it to be both the first and last test in B. Inter-class test pairs that were already covered in the initial intra-class orders are skipped. Once all inter-class test-pairs are covered for a given class order, TuscanE begins covering inter-class test-pairs in another Tuscan permutation of test classes.

To cover intra-class and class pairs (Step 1), TuscanE

uses the algorithms from prior work [20], with the only difference being that TuscanE records a list of inter-class test-pairs that are covered, and it later uses this list to skip duplicates. Figure 3 shows the TuscanE algorithm for generating test-orders that cover all inter-class test-pairs (Step 2). It takes as input a list of already covered test-pair (`alreadyCoveredInterclasspair`), so TuscanE does not re-run inter-class test-pairs that were already covered. The algorithm begins by sorting the classes in alphabetical order (Line 2). If there is only one class, it ends early as there are no inter-class test-pairs to cover (lines 3–4). For multiple classes, it generates a Tuscan class order matrix (Line 5) to arrange the classes across different test-orders. The loop from lines 7–27 goes through each row of this matrix (excluding the last element in each row). For every adjacent pair of classes, Line 10 generates all possible test-pairs. The largest number of test-pairs across all adjacent class pairs (denoted as T in Line 14) decides how many test-orders will be created from each row. In lines 15–30, a complete test-order is built for each iteration: the first class’s single test remains fixed (lines 17–20), tests from the middle classes that are needed for new inter-class test-pairs (lines 21–26), and the last class’s single test is selected based on the last test-pair a new test-order is expected to cover (lines 27–30). By repeating this process for every iteration, the algorithm guarantees that each test is eventually paired with every test at least once. While generating the test-orders, if any of the test-pairs are in `alreadyCoveredInterclasspair`, TuscanE removes them from its test-orders in Line 13.

B. Tuscan inter-class Approach on Generating Test-Orders

The middle portion of Figure 2 shows how the Tuscan inter-class approach proposed by Li et al. [20] works on the same example described in Section III-A. Unlike TuscanE, Tuscan inter-class does not separate the generation of test-orders that

cover intra-class test-pairs from test-orders that cover inter-class test-pairs. Instead, it immediately computes the Tuscan permutations at the class-level, e.g., starting with $(A \rightarrow B \rightarrow C)$. Then, for each pair of adjacent test classes, it separately computes a set of permutations of the tests within each test class based on Tuscan squares. Tuscan inter-class then pairs each permutation between the two adjacent test classes against each other, computing all possible combinations of the permutations between the two adjacent test classes to construct test-orders. For adjacent test classes A and B, it computes separately the permutation of tests in each test class to generate test-orders that pair them against each other. In doing so, inter-class test-pairs involving A and B naturally get covered, since Tuscan squares guarantees that each test is positioned first in one permutation and last in another one. However, we see that each test-order covers only one additional inter-class test-pair (instead of as many as possible). As such, Tuscan inter-class is quite inefficient and generates many more test-orders than needed. For this example, Tuscan Inter-class generates 64 test-orders, running 512 tests in the end, compared to the 28 test-orders and 94 tests needed by TuscanE. In short, TuscanE differs from Tuscan inter-class in how it (1) maximizes the coverage of inter-class test pairs in each order, (2) prioritizes more important intra-class orders to be covered first, and (3) removes tests from orders to cover only unique test pairs.

C. Pairwise Approach on Generating Test-Orders

The right side of Figure 2 shows the Pairwise approach discussed in prior work [13], [24]. Each test is paired with every other test (skipping the case where a test is paired with itself) and each pair of tests is run twice – once where a test is run as the first element of the pair (with the remaining test as the second element) and a second time, where the same test is run as the second element of the pair (with the remaining test as the first element). Every test-order then has exactly two tests, corresponding to each test-pair; each test-order covers only one test-pair and the test-orders cannot cover additional test-pairs with adjacent tests within a single longer sequence, which is why Pairwise typically needs to run more test-orders (and tests) than TuscanE. In fact, in this example, Pairwise generates 56 test-orders and runs 112 tests compared to the 28 test-orders and 94 tests needed by TuscanE.

D. Number of generated test-orders

To better understand how TuscanE compares with Tuscan inter-class [20] and Pairwise [13], we estimate the number of orders they need to achieve 100% coverage of all test-pairs.

1) *TuscanE Approach:* We can compute the upper-bound number of test-orders generated using TuscanE as:

$$\max(|Y|, \max_{y_i \in Y}(|y_i|)) + \sum_{r=1}^{|Y|} \max_{0 \leq j < |Y|-1} (|y_{T[r][j]}| \times |y_{T[r][j+1]}|) \quad (1)$$

where $|Y|$ represents the number of test classes, $|y_i|$ represents the number of tests in test class y_i , and $\max(|Y|, \max_{y_i \in Y} |y_i|)$ represents the larger value between

```

1 def TuscanEInterClassOrders(alreadyCoveredInterclasspair):
2     classes = sorted(classToMethods.keys())
3     if len(classes) == 1:
4         return singleOrderForSingleClass(classes[0])
5     co = genTuscanPermutations(len(classes))
6     allOrders = []
7     for r in range(len(co)):
8         orderedClasses = [classes[co[r][i]] for i in
9                             range(len(co[r]) - 1)]
10        # Build lex lists for adjacent class pairs
11        lexList = [ [x, y] for x in range(1,
12                                len(classToMethods[orderedClasses[i]] + 1)
13                                for y in range(1,
14                                len(classToMethods[orderedClasses[i+1]] + 1) )
15                    for i in range(len(orderedClasses)-1) ]
16        lexList = [ [p for p in lst if (orderedClasses[i],
17                                p[0], orderedClasses[i+1], p[1]) not in
18                                alreadyCoveredInterclasspair] ] # remove
19                                already-covered inter-class pairs
20        T = max([len(l) for l in lexList])
21        for k in range(T):
22            order = []
23            # Process 1st class
24            mf = classToMethods[orderedClasses[0]]
25            pf = lexList[0][k % len(lexList[0])]
26            order.extend([mf[0]] if mf[0] == mf[pf[0]-1] else
27                        [mf[0], mf[pf[0]-1]])
28            # Process mid classes
29            for i in range(1, len(orderedClasses)-1):
30                mc = classToMethods[orderedClasses[i]]
31                pp = lexList[i-1][k % len(lexList[i-1])]
32                cp = lexList[i][k % len(lexList[i])]
33                order.extend([mc[pp[1]-1]] if mc[pp[1]-1] ==
34                            mc[cp[0]-1] else [mc[pp[1]-1], mc[cp[0]-1]])
35            # Process last class
36            ml = classToMethods[orderedClasses[-1]]
37            lp = lexList[-1][k % len(lexList[-1])]
38            order.extend([ml[lp[1]-1]] if ml[lp[1]-1] == ml[0]
39                        else [ml[lp[1]-1], ml[0]])
40            allOrders.append(order)
41        return allOrders

```

Fig. 3: TuscanE pseudo-code to generate inter-class test orders.

the number of test classes and the maximum number of tests in any class. T represents a Tuscan square (or Tuscan permutation) matrix of size $|Y| \times |Y|$, $T[r][j]$ represents the index corresponding to the test class placed in the j -th position of row r in T , $y_{T[r][j]}$ represents the actual test class corresponding to index $T[r][j]$, $\max_{0 \leq j < |Y|-1}$ represents the maximum product of test counts across all adjacent test class pairs in a class row r , and $\sum_{r=1}^{|Y|}$ represents the summation for all maximum values for each of the $|Y|$ rows.

TuscanE guarantees intra-class test-pair coverage by generating at least as many test-orders as the larger of the number of test classes and the maximum number of tests in any class. This guarantee is because TuscanE needs enough test-orders to cover both the Tuscan square over classes and the full Tuscan square for the class with the most tests, which corresponds to the first term in Equation 1. The second term calculates the number of test-orders needed for inter-class test-pair coverage. For each Tuscan row, it considers the maximum product of tests counts between any adjacent class pair, representing the number of test-orders required to cover inter-class test-pairs between those two classes, assuming that test-orders for other class pairs can be concurrently covered in the same iteration.

Note that this equation is strictly an upper-bound, as we are unable to model the complexity introduced by using `alreadyCoveredInterclasspair`, which would reduce the number of generated test-orders. Considering the example

from Figure 2, this equation computes an upper-bound that is still lower than the number generated by other approaches.

2) *Tuscan inter-class Approach*: The number of test-orders generated by Tuscan inter-class is:

$$\sum_{(r,j) \in \mathcal{I}} (|y_{T[r][j]}| \times |y_{T[r][j+1]}|) \quad (2)$$

where $\mathcal{I} := \{(r,j) \mid 1 \leq r \leq |Y|, 0 \leq j < |Y| - 1\}$ is the index set of all adjacent class-pair positions across all Tuscan rows, and $\sum_{(r,j) \in \mathcal{I}}$ denotes summation of the products $|y_{T[r][j]}| |y_{T[r][j+1]}|$ over all such adjacent pairs. Equation 2 sums the product of test counts for *every* adjacent class pair for all Tuscan class rows. Doing so relies on brute force to create the combinations.

Tuscan inter-class [20] produces fewer test-orders than TuscanE in only two cases: (1) when there are exactly two test classes ($|Y| = 2$) regardless of any number of tests in the two classes, or (2) when only one or two test classes have more than one test per class (i.e., $|\{i \mid |y_i| > 1\}| \leq 2$). In case (1), each Tuscan row contains a single adjacent pair, so the inter-class portion on the right of Equation 1 becomes the same as the full sum of adjacent products in Equation 2. As Equation 1 also adds the intra-class portion $\max(|Y|, \max_{y_i \in Y} |y_i|)$, TuscanE creates an additional $\max_{y_i \in Y} |y_i|$ test-orders. In case (2), all test classes with just one test do not affect the number of orders generated, therefore if only two classes have more than one test, then this case just reduces to be the same as case (1). Similar to case (1), the intra-class term $\max(|Y|, \max_{y_i \in Y} |y_i|)$ of Equation 1 ends up adding $\max_{y_i \in Y} |y_i|$ test-orders to TuscanE that Tuscan inter-class would not have.

3) *Pairwise Approach*: A pairwise approach will produce $N \times (N - 1)$ orders, where N is the number of tests. The count for Pairwise depends only on the number of tests $N = \sum_{i=1}^{|Y|} |y_i|$, and which tests are in which classes does not affect the number of orders. A Pairwise approach generates every possible two-test sequence of distinct tests, expressed as:

$$N \times (N - 1) = \left(\sum_{i=1}^{|Y|} |y_i| \right) \left(\sum_{i=1}^{|Y|} |y_i| - 1 \right) \quad (3)$$

As TuscanE can generate longer test-orders (length ≥ 3), each test-order can cover multiple adjacent test-pairs, reducing the number of test-orders and tests. For example, a Tuscan row $A.1 \rightarrow B.1 \rightarrow C.1$ covers two ordered pairs, $(A.1, B.1)$ and $(B.1, C.1)$, whereas Pairwise needs two separate test-orders and four tests. Thus, TuscanE produces fewer test-orders whenever $N > 2$; when $N \leq 2$, there are not enough tests to share a test across two adjacent pairs, so TuscanE ties with Pairwise but is never worse.

E. Number of tests run

Formally, let Y be the set of test classes with $|y_i|$ tests in class y_i , and let $N := \sum_{i=1}^{|Y|} |y_i|$ be the number of tests. The number of tests executed by Tuscan inter-class [20] is:

$$N \times \sum_{(r,j) \in \mathcal{I}} (|y_{T[r][j]}| \cdot |y_{T[r][j+1]}|) \quad (4)$$

where $\sum_{(r,j) \in \mathcal{I}} |y_{T[r][j]}| |y_{T[r][j+1]}|$ is the number of test-orders from Equation 2.

TuscanE not only generates fewer test-orders than the Tuscan inter-class in most cases, but TuscanE also run fewer tests by tracking inter-class pairs during the intra-class phase to avoid re-running pairs that are already covered and skipping tests that do not contribute to not covered pairs. As a result, TuscanE runs fewer tests than Tuscan inter-class in all subjects of our evaluation (Tables I and II). In fact, the total tests ran by TuscanE is never greater and is often the same or smaller.

The total number of tests run by Pairwise is:

$$2 \times N \times (N - 1) \quad (5)$$

Compared to Pairwise, TuscanE always runs *fewer or the same* number of tests. TuscanE ties only when there is at most two total tests ($N \leq 2$), because no sharing of adjacent pairs is possible. In all other cases, TuscanE will run fewer tests.

IV. EVALUATION SETUP

Our evaluation studies the following research questions:

RQ1: How efficient are test-orders generated by TuscanE at covering all test-pairs?

RQ2: What is the effect of removing duplicate-pair tests from test-orders?

RQ3: How effective is TuscanE at generating test-orders to detect the first OD test?

We address RQ1 to evaluate the efficiency of the test-orders generated by TuscanE while ensuring all test-pairs are covered. Generating fewer test-orders is preferable, as it reduces the number of times a developer needs to execute the tests. Generating fewer tests is also preferable as it reduces the overall test run time. Additionally, generation time is also an important factor, as shorter generation times allows developers to begin running those test-orders sooner and to quickly regenerate test-orders when the test suite changes. We address RQ2 to evaluate how removing all duplicate-pair tests, only duplicate-pair tests in inter-class test-orders, or not removing duplicate tests affects the efficiency of orders generated by TuscanE. This ablation study helps explain when removing tests is beneficial and when it can be harmful. For RQ3, we evaluate how quickly different techniques detect the first OD test. A developer may begin debugging once an OD test is detected, which requires at least one passing and at least one failing outcome across different test-order executions.

Dataset. To evaluate TuscanE, we utilize a publicly available dataset containing OD tests from 46 GitHub Maven modules¹, identified in previous work by Li et al. [20]. Table I presents a detailed breakdown and statistics for all modules we use in our evaluation (detailed names are on our website [25]). We also show the average test suite runtime from running the test suite 10 times. Following prior work [20], we ran our experiments on the same dataset to enable a direct comparison, but we exclude the module “fastjson”, as Tuscan inter-class could not finish generating test-orders within the 12-hour time limit (although TuscanE and Pairwise could), so we could not conduct a fair comparison. Note that TuscanE is dataset-agnostic: it can be applied to any project, since it requires

¹A Maven project may contain multiple modules, each having a test suite.

TABLE I: Subjects overview. “Runtime Test Suite” is the time to run all tests once. “Runtime Overhead” is the overhead time to setup/teardown an order, regardless of how many tests are in the order. “Generation time” columns show the time all approaches take to generate (but not run) orders needed to cover all pairs.

ID	Module	Number of		Runtime (in sec.)		Generation time (in sec.)		
		Tests	Test Classes	Test Suite	Overhead	TuscanIC	Pairwise	TuscanE
M1	activiti-spring-boot-starter	45	11	0.50	5.51	0.80	0.04	0.11
M2	aismessages	44	18	0.16	1.08	1.46	0.03	0.10
M3	c2mon-server-elasticsearch	19	11	10.27	13.94	0.67	0.01	0.08
M4-1	compute	488	74	143.17	4.40	521.43	1.20	2.10
M4-2	request	325	49	289.13	8.59	208.81	1.40	1.15
M5	cukes-http	14	4	0.76	1.44	0.67	0.00	0.08
M6	dropwizard-logging	80	18	3.76	2.14	1.28	0.11	0.20
M7-1	dubbo-cluster	112	23	3.99	1.81	2.28	0.24	0.35
M7-2	dubbo-common	520	70	31.76	1.62	112.25	1.46	3.61
M7-3	dubbo-config-api	263	23	4.92	2.24	13.65	0.33	0.93
M7-4	dubbo-filter-cache	9	5	2.96	1.66	0.57	0.01	0.07
M7-5	dubbo-rpc-api	60	23	3.75	1.51	0.98	0.06	0.13
M7-6	dubbo-rpc-dubbo	74	16	159.64	41.40	1.16	0.10	0.16
M7-7	dubbo-serialization-fst	21	4	0.30	1.38	0.62	0.01	0.10
M8	elastic-job-lite-core	511	98	32.09	63.48	137.45	1.67	1.96
M9-1	hadoop-auth	125	22	100.03	4.10	3.01	0.27	0.35
M9-2	hadoop-hdfs-nfs	64	12	43.88	6.35	0.96	0.08	0.17
M9-3	hadoop-mapreduce-client-app	394	49	289.48	16.00	61.79	1.08	2.16
M9-4	hadoop-mapreduce-client-core	293	60	78.91	5.09	28.52	0.54	1.29
M9-5	hadoop-mapreduce-client-hs	206	26	68.45	8.06	11.76	0.36	0.63
M10	hsac-fitness-fixtures	251	49	7.30	3.73	14.20	0.72	0.67
M11-1	integration-test-2_1	233	40	18.95	18.38	12.55	0.56	0.91
M11-2	integration-test-2_2	23	4	1.92	13.95	0.74	0.01	0.09
M11-3	integration-test-3_10	24	3	0.82	11.49	0.72	0.01	0.09
M11-4	integration-test-3_7	6	1	0.51	11.84	0.57	0.00	0.08
M12	jhipster-registry	53	14	1.49	8.63	0.94	0.05	0.14
M13	lib	163	2	0.23	1.34	0.92	0.18	0.14
M14	marine-api	926	71	0.63	1.44	1,089.97	0.62	4.15
M15	openpojo	1,185	213	11.69	2.80	6,566.07	2.22	8.71
M16	org.wso2.carbon.apimgt.core	530	46	76.11	8.44	616.40	0.07	10.39
M17	portlet	61	12	1.17	1.65	1.03	0.09	0.15
M18	riptide-spring-boot-starter	40	25	1.47	6.85	1.64	0.20	0.12
M19-1	spring-boot	2,255	230	27.85	13.27	23,625.56	13.03	80.42
M19-2	spring-boot-actuator-autoconfigure	733	166	24.13	13.57	1,997.08	1.59	4.61
M19-3	spring-boot-test	721	115	2.32	12.89	1,764.98	3.97	16.25
M19-4	spring-boot-test-autoconfigure	234	103	3.03	59.40	50.91	0.04	0.71
M20	spring-data-ebean	48	2	0.30	3.17	0.76	0.00	0.11
M21	spring-data-envers	10	4	0.60	3.88	1.31	0.22	0.09
M22-1	spring-ws-core	971	145	8.12	5.70	3,216.01	0.26	4.99
M22-2	spring-ws-security	122	43	1.24	1.91	6.04	0.01	0.20
M23-1	naming	82	11	1.27	2.26	3.12	0.18	0.22
M23-2	subsystem	25	10	3.76	2.33	2.06	0.07	0.10
M24	unix4j-command	290	30	4.38	0.89	48.02	0.05	1.00
M25-1	wdtk-dumpfiles	50	9	2.59	4.30	1.54	0.01	0.14
M25-2	wdtk-util	23	5	0.46	1.33	2.03	0.12	0.09
M26	wro4j-core	851	137	9.58	1.12	2,424.08	1.61	5.79
Average		295	46	32.17	8.88	925.20	0.76	3.39

only a single test-order as input and to generate a set of test-orders needed to achieve 100% coverage of test-pairs. As prior work [20] already identified OD tests and their associated polluters, cleaners, and state setters, we simulate detection of OD tests by a generated test-order without running it; a victim fails if a polluter runs before with no cleaner in-between, and a brittle fails if no state setter runs before. The OD test passes otherwise. An OD test is detected when it passes at least once and fails at least once in different test-orders. For Random generation in RQ3, we generate random test-orders 10 times and report the average number of test-orders and time needed to detect the first OD test.

Metrics. To evaluate RQ1 and RQ2, we consider: the number of test-orders, the number of tests generated, and the time required to generate them. Fewer test-orders mean developers run the test suite fewer times to detect all OD tests, speeding

up detection; fewer tests mean each run is faster, further saving time. Generation time is also critical: when tests are added or removed, developers must regenerate test-orders, so we measure how long generation takes. However, generation time alone does not determine an approach’s overall value, some methods generate quickly but produce many more test-orders or tests, increasing the total detection time. Therefore, we introduce a combined metric, *time to generate and run* for RQ1 and RQ2. For RQ2 and RQ3, we measure first-OD test detection using the number of test-orders needed to detect the first OD test and the time needed to detect the first OD test.

Baseline Comparison. We compare our TuscanE approach against Tuscan inter-class [20], Pairwise [13] and random test-order generation [2], [13]–[15]. For guaranteeing complete test-pair coverage (RQ1 and RQ2), we compare specifically against Tuscan inter-class [20] and Pairwise, as random gen-

eration cannot provide such guarantees. As for first OD test detection (RQ3), we compare TuscanE against all baselines. Our website [25] contains scripts to use all approaches.

Experiment Configurations. We set TuscanE and other compared approaches to generate test-orders at most 12 hours per module, reflecting a realistic scenario in which developers can generate test-orders overnight. All experiments were conducted on a machine with an Intel Core i7-9700 CPU at 3.00 GHz, 32 GB RAM, running Ubuntu 22.04.2 LTS.

V. EVALUATION RESULTS

RQ1: Efficiency of TuscanE test-orders. Tables I and II present a comparison between TuscanE, Tuscan inter-class, and Pairwise in terms of their efficiency generating test-orders and how efficient the generated test-orders are at covering all test-pairs. Table II shows that Tuscan inter-class generates 7.95 times more test-orders than TuscanE, while Pairwise generates 8.15 times more test-orders. TuscanE also generates fewer test-orders than Tuscan inter-class in 43 out of 46 modules and fewer test-orders than Pairwise in all modules. In the three modules where TuscanE generates the same number of test-orders as Tuscan inter-class (M11-4, M13, and M20), the modules have only one or two test classes, meaning there are few inter-class test-pairs. For the number of tests run, Table II shows that Tuscan inter-class runs 152.74 times more tests than TuscanE, while Pairwise runs 1.06 times more tests.

TuscanE also demonstrates efficiency in actual time taken to generate test-orders and run them. The “Generation time (in sec.)” column in Table I shows the time each approach needed to generate test-orders, while the “Time to generate and run” column in Table II summarizes the overall performance of the approaches, which includes time to generate and run test-orders. We see that even though TuscanE took 1.54 seconds more than Pairwise to generate the test-orders, Pairwise takes on average 7.21 times more time to generate and run the test-orders (372,967 seconds for TuscanE vs 2,687,613 seconds for Pairwise). In fact, for all modules, Pairwise took more time than TuscanE. When we compare the total time taken by TuscanE and Tuscan inter-class, we find that Tuscan inter-class takes 26.75 times more time than TuscanE (9,976,133 seconds for Tuscan inter-class vs 372,967 seconds for TuscanE). Overall, the total time to generate and run test-orders to cover all test-pairs and detect all OD tests is substantially lower for TuscanE compared to Tuscan inter-class and Pairwise.

RQ2: Effect of removing tests from test-orders. By default, we configure TuscanE to remove tests that do not cover new pairs only during inter-class test-order generation. Table III compares TuscanE with two ablated configurations: (1) removing all (*R. All*) non-contributing tests from both intra-class and inter-class test-orders and (2) keeping the full test suite in each test-order and does not remove any (*R. None*) tests.

TuscanE detects the first OD test after only 6.67 test-orders and 196.06 seconds on average. In comparison, *R. All* requires 17.02 test-orders and 258.89 seconds, while *R. None* requires 7.48 test-orders and 244.06 seconds. These results show that removing tests during the intra-class phase can hurt first-OD

test detection. Covering class pairs during the intra-class phase helps detect victims and brittles whose polluters, cleaners, or state setters are in different classes. In such cases, class-level pair coverage may be sufficient to quickly expose an OD test early. For example, in M6, *R. All* requires 454 test-orders to detect the first OD test, while TuscanE requires only 15 test-orders; similarly, in M7-5, *R. All* requires 44 test-orders, while TuscanE requires only 7 test-orders. In both modules, the polluter, cleaner, and victim are in different classes, so removing tests during the intra-class phase removed tests needed to expose the OD test early.

The comparison with *R. None* shows the other side of the tradeoff: keeping full-suite test-orders can preserve tests needed to expose an OD test, but it also runs many tests that do not help cover new test-pairs. In some modules, such as M6, M7-5, M9-5, and M14, *R. None* can require more test-orders to detect the first OD test because full-suite test-orders cannot always share inter-class test-pair coverage as effectively as TuscanE. Namely, a given test from a test class with multiple tests cannot be both the first and last test of the test class if the other tests in the class must run. Thus, TuscanE may cover both test-pairs in one test-order, while *R. None* may need two separate test-orders. This limitation can hinder TuscanE’s ability to as quickly as possible cover inter-class test-pairs.

Intuitively, the default TuscanE, which removes non-contributing test-pairs only in inter-class generation, is going to have a worse overall runtime than *R. All* (374,290 seconds vs. 373,159 seconds for *R. All*) as these two configurations would have the same number of test-orders, but *R. All* would run fewer total tests. Despite not making *R. All* the default configuration for TuscanE, we still find our default TuscanE to substantially outperform all baselines in RQ1. More importantly, the improvement for first OD detection in our default TuscanE configuration from *R. All* will matter in RQ3.

RQ3: Effectiveness of TuscanE test-orders at detecting the first OD test. Table III shows that the test orders generated by TuscanE are more effective at detecting the first OD test than those generated by the compared approaches. On average, TuscanE detects the first OD test after only 6.67 test orders, while a random-order-based approach, Tuscan inter-class, and Pairwise require 8.22, 16680.70, and 111082.74 test orders, respectively. Similarly, TuscanE detects the first OD test in less time than the other approaches.

To understand whether TuscanE significantly improves over the best baseline, random, we performed a paired Wilcoxon signed-rank test. For first-OD test detection order, TuscanE requires fewer test-orders than Random in 38 out of 46 modules ($W = 262.0$, one-sided $p = 0.00117$; two-sided $p = 0.00234$). For first-OD test detection time, TuscanE is faster than Random in 38 out of 46 modules ($W = 276.5$, one-sided $p = 0.00196$; two-sided $p = 0.00392$). These results show that TuscanE’s improvement over random appears consistent across modules. Overall, our results show that TuscanE detects the first OD test earlier than all baseline approaches and that systematic test-pair coverage can be useful for detecting first OD test faster.

TABLE II: Comparison of all approaches. “Time to generate and run” is the sum of the “Generation time” in Table I and the time to run all orders and tests. TuscanIC is Tuscan inter-class [20].

ID	Number of test orders run			Number of tests run			Time to generate and run (in sec.)		
	TuscanIC	Pairwise	TuscanE	TuscanIC	Pairwise	TuscanE	TuscanIC	Pairwise	TuscanE
M1	1,794	1,980	438	80,730	3,960	3529	10,780	10,948	2439
M2	1,788	1,892	361	78,672	3,784	3346	2,228	2,059	389
M3	312	342	97	5,928	684	564	7,555	5,138	1607
M4-1	232,368	237,656	26090	113,395,584	475,312	467229	34,290,315	1,185,132	245421
M4-2	101,578	105,300	16497	33,012,850	210,600	203914	30,242,436	1,091,992	315828
M5	122	182	87	1,708	364	284	269	282	140
M6	5,640	6,320	1835	451,200	12,640	11565	33,286	14,107	4449
M7-1	11,244	12,432	3682	1,259,328	24,864	23070	65,214	23,363	7454
M7-2	261,418	269,880	47626	135,937,360	539,760	530343	8,726,717	470,176	108196
M7-3	64,268	68,906	13906	16,902,484	137,812	133277	459,811	156,583	33483
M7-4	70	72	30	630	144	117	324	167	91
M7-5	3,386	3,540	580	203,160	7,080	6390	17,810	5,774	1218
M7-6	5,036	5,402	958	372,664	10,804	9895	1,012,403	246,928	59826
M7-7	224	420	147	4,704	840	678	379	593	218
M8	256,130	260,610	24619	130,882,430	521,220	509234	24,479,926	16,577,300	1588039
M9-1	14,270	15,500	3820	1,783,750	31,000	28797	1,485,985	88,374	37696
M9-2	3,330	4,032	1539	213,120	8,064	7339	167,288	31,149	14534
M9-3	148,542	154,842	29147	58,525,548	309,684	300637	45,376,463	2,704,849	679712
M9-4	82,814	85,556	17984	24,264,502	171,112	163728	6,956,601	481,905	133378
M9-5	39,500	42,230	9329	8,137,000	84,460	79898	3,022,131	368,397	101044
M10	60,816	62,750	8900	15,264,816	125,500	121054	671,192	237,898	36471
M11-1	50,658	54,056	13177	11,803,314	108,112	101902	1,891,165	1,002,451	249838
M11-2	382	506	197	8,786	1,012	860	6,062	7,144	2818
M11-3	316	552	222	7,584	1,104	892	3,893	6,382	2695
M11-4	6	30	6	36	60	36	75	360	74
M12	2,572	2,756	455	136,316	5,512	4990	26,032	23,937	4041
M13	644	26,406	644	104,972	52,812	27209	1,012	35,406	1113
M14	841,810	856,550	50739	779,516,060	1,713,100	1686655	1,750,820	1,238,033	74362
M15	1,392,896	1,403,040	86349	1,650,581,760	2,806,080	2769201	20,196,687	3,959,009	267457
M16	237,882	280,370	124083	126,077,460	560,740	545507	20,112,647	2,446,564	1119554
M17	3,306	3,660	842	201,666	7,320	6601	9,320	6,169	1500
M18	1,512	1,560	256	60,480	3,120	2534	12,584	10,805	1745
M19-1	5,015,642	5,082,770	612922	11,310,272,710	10,165,540	10131892	206,283,212	67,594,247	8253310
M19-2	531,042	536,556	49282	389,253,786	1,073,112	1054367	20,022,901	7,314,789	699463
M19-3	472,332	519,120	171633	340,551,372	1,038,240	1003477	7,186,061	6,692,731	2215427
M19-4	53,474	54,522	7491	12,512,916	109,044	99931	3,338,344	3,240,018	442828
M20	574	2,256	574	27,552	4,512	3034	1,995	7,189	1948
M21	58	90	38	580	180	136	261	360	171
M22-1	932,108	941,870	56126	905,076,868	1,883,740	1861797	12,885,850	5,380,654	333845
M22-2	14,452	14,762	1063	1,763,144	29,524	27022	45,507	28,423	2210
M23-1	5,712	6,642	2281	468,384	13,284	12225	20,179	15,230	5329
M23-2	554	600	121	13,850	1,200	1001	3,377	1,579	410
M24	79,464	83,810	14716	23,044,560	167,620	162313	419,003	76,957	15442
M25-1	2,116	2,450	719	105,800	4,900	4341	14,585	10,791	3320
M25-2	492	506	174	11,316	1,012	876	882	691	250
M26	713,822	723,350	62889	607,462,522	1,446,700	1432725	7,640,534	827,162	85720
Average	253,228	259,536	31,841	363,040,043	519,071	511879	9,976,133	2,687,613	372,967

VI. THREATS TO VALIDITY

We obtained our results by using a prior OD test dataset [19] drawn from popular open-source Java projects (also used in subsequent work [20]), which may not generalize to other subjects. TuscanE operates with the constraint of no inter-class test interleaving in JUnit testing framework, which would not be relevant in testing frameworks or languages that allow that. In those cases, it would suffice to use basic Tuscan squares [19]. All the RQs compare run time and generation time, which require per-module test-suite runtime and overhead; we obtained these by averaging 10 runs per module. Although the number of runs is limited, the measured runtimes consistently influenced all approaches in the same direction. Finally, the generation times in Table I, Table II, and Table III can vary with environment configuration, which may affect the time differences observed between approaches.

VII. RELATED WORK

Flaky tests have been extensively studied, with them appearing in both open-source software and industry code [1], [6], [9], [10], [14], [31]. Luo et al. found that OD tests are among the top-three most common types of flaky tests [1]. Zhang et al. [13] and Lam et al. [2] developed random-based test order generation techniques to detect OD tests.

These prior techniques rely on randomizing the test-orders, which do not provide strong guarantees on whether the OD test can be detected. Wei et al. proposed using Tuscan squares to guide the generation of test-orders, providing better guarantees on detecting OD tests [19]. Li et al. developed Tuscan Square inspired variations based around JUnit and Surefire restrictions on valid test-orders [20]. Our work improves upon Li et al.’s work by generating more effective and efficient test-orders.

Bell et al. proposed ElectricTest [17], which Gambi et

TABLE III: First OD test detection results for *R. All* (TuscanE but removes all non-contributing tests), *R. None* (TuscanE but does not remove any tests), Random, Pairwise, Tuscan inter-class, and TuscanE.

ID	Numbers of orders to detect the 1st OD						Time (in sec.) to detect the 1st OD					
	<i>R. All</i>	<i>R. None</i>	Random	TuscanIC	Pairwise	TuscanE	<i>R. All</i>	<i>R. None</i>	Random	TuscanIC	Pairwise	TuscanE
M1	2	2	2.20	19	37	2	12.02	12.02	13.22	114.17	227.98	12.02
M2	2	2	3.10	9	290	2	2.49	2.49	3.86	11.21	395.90	2.49
M3	2	2	5.10	20	24	2	48.43	48.43	123.49	484.31	418.67	48.43
M4-1	11	11	22.80	2071	8697	11	1623.24	1623.23	3364.52	305615.42	50186.40	1623.24
M4-2	2	2	5.10	1270	1522	2	595.45	595.45	1518.39	378112.33	16497.17	595.45
M5	2	2	3.90	3	156	2	4.40	4.40	8.58	6.62	470.63	4.40
M6	454	17	17.10	267	2552	15	2679.39	100.33	100.92	1575.79	6542.41	88.53
M7-1	2	2	2.30	2	714	2	11.60	11.60	13.34	11.60	5482.79	11.60
M7-2	5	6	3.40	3781	168198	5	166.91	200.29	113.50	126218.22	300452.92	166.91
M7-3	2	2	17.80	23	31	2	14.31	14.31	127.35	164.56	89.73	14.31
M7-4	2	2	3.20	8	34	2	9.25	9.26	14.79	37.05	118.15	9.26
M7-5	44	8	8.20	432	28	7	231.44	42.08	43.13	2272.31	48.06	36.82
M7-6	3	4	4.80	53	489	3	603.10	804.13	964.96	10654.76	27916.88	603.10
M7-7	2	2	4.10	9	48	2	3.38	3.37	6.92	15.21	85.77	3.38
M8	4	5	2.10	666	15095	4	382.30	477.88	200.71	63653.73	968851.91	382.30
M9-1	8	9	5.10	3315	14236	7	833.07	937.20	531.08	345202.60	84297.14	728.93
M9-2	2	2	2.70	282	88	2	100.47	100.47	135.64	14166.74	711.18	100.47
M9-3	2	2	2.40	2689	14588	2	610.96	610.96	733.15	821433.05	258144.08	610.96
M9-4	2	2	2.80	1084	5244	2	168.00	168.00	235.21	91058.94	30350.22	168.00
M9-5	10	11	2.50	13474	11602	10	765.09	841.60	191.27	1030890.97	101869.61	765.09
M10	7	8	7.20	665	33107	7	77.25	88.29	79.46	7339.23	129361.08	77.25
M11-1	2	2	4.30	612	493	2	74.66	74.66	160.53	22847.19	9202.66	74.66
M11-2	2	2	4.60	29	2	2	31.74	31.74	72.99	460.24	30.42	31.74
M11-3	2	2	3.60	2	93	2	24.63	24.63	44.34	24.64	2220.70	24.64
M11-4	2	2	4.00	3	2	2	24.73	24.70	49.40	37.33	40.49	24.75
M12	2	2	3.40	29	852	2	20.24	20.24	34.41	293.51	7994.56	20.24
M13	2	2	2.60	3	915	2	3.14	3.14	4.08	4.71	2184.20	3.14
M14	42	46	29.50	12553	57372	42	87.30	95.61	61.32	26108.08	83322.78	87.30
M15	3	4	3.80	5204	153118	3	43.49	57.98	55.08	75456.86	433337.91	43.49
M16	2	20	2.60	59	179302	2	169.09	1690.92	219.82	4988.38	2078499.57	169.09
M17	2	2	2.70	242	117	2	5.64	5.64	7.61	682.21	199.92	5.64
M18	2	2	2.90	2	60	2	16.65	16.64	24.13	16.65	914.89	16.64
M19-1	33	33	22.30	708370	4215579	33	1357.07	1357.07	917.05	29133825.48	56069796.55	1357.07
M19-2	2	2	2.50	329	139418	2	75.40	75.40	94.25	12404.92	1932617.45	75.40
M19-3	2	2	3.40	953	8585	2	30.42	30.42	51.71	14498.95	110955.70	30.42
M19-4	2	2	2.60	6	10455	2	124.86	124.86	162.31	374.58	838860.35	124.86
M20	7	8	57.00	294	32	7	24.32	27.79	198.03	1021.82	104.62	24.32
M21	15	18	15.40	3	6	15	67.24	80.62	68.98	13.50	158.36	63.29
M22-1	42	42	9.10	950	3529	42	580.48	580.48	125.77	13133.20	22316.58	580.48
M22-2	27	28	33.70	336	201	28	85.01	88.16	106.10	1058.01	439.74	88.16
M23-1	2	2	2.30	161	6	2	7.07	7.06	8.12	568.77	14.02	7.06
M23-2	2	2	17.70	160	4	2	12.18	12.18	107.82	975.22	10.83	12.18
M24	4	4	3.10	1624	25632	4	21.09	21.09	16.34	8563.14	23869.09	21.09
M25-1	8	8	11.20	5	36	8	55.14	55.14	77.19	34.46	555.56	55.14
M25-2	2	2	3.90	2	358	2	3.58	3.58	6.98	3.59	1129.40	3.58
M26	2	2	2.20	5239	36859	2	21.40	21.40	23.54	56076.66	42299.31	21.40
Average	17.02	7.48	8.22	16,680.70	111,082.74	6.67	258.89	244.06	243.94	708,098.06	1,383,556.40	196.06

al. later enhanced to PraDet [18], to detect dependencies between tests analyzing reads/writes to shared data. Gyori et al. developed PolDet to detect when tests may pollute shared state, potentially resulting in OD tests [32]. Rahman et al. developed RankF [33], which detects tests that OD tests depend on. In contrast, we focus on generating an efficient set of test-orders to detect all OD tests, without analyzing shared state or data. Lam et al. [34] proposed to cope with OD tests by adding or reordering tests to satisfy test-order dependencies. Their work requires (1) developers to compute dependencies when code changes and (2) the use of suboptimal regression testing technique orders as orders are modified to satisfy dependencies.

Combinatorial testing is the process of systematically combining input parameters in software as to more efficiently expose faults that are due to interactions between these parameters [35]. There has been work in how to more efficiently

construct these combinations to achieve t-wise coverage, i.e., covering all t-tuples of parameters [36], [37]. Unlike combinatorial test, our work aims to systematically generate *permutations* of test-orders to detect OD test.

VIII. CONCLUSION

Detecting OD tests is costly because guaranteeing detection typically requires running many test-orders. We proposed TuscanE, a two-step systematic test-order generator that achieves 100% test-pair coverage while generating fewer test-orders and running fewer tests. Across our subjects, TuscanE outperforms Tuscan inter-class and Pairwise for all-OD test detection by reducing the total time to generate and run all test-orders. TuscanE also detects the first OD test earlier than Random, Tuscan inter-class, and Pairwise on average. In practice, developers can use TuscanE to systematically generate efficient test-orders for both all-OD test and first-OD test detection.

REFERENCES

- [1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests,” in *FSE 2014: Proceedings of the ACM SIGSOFT 22nd Symposium on the Foundations of Software Engineering*, 2014.
- [2] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, “iDFlakies: A framework for detecting and partially classifying flaky tests,” in *ICST 2019: 12th International Conference on Software Testing, Verification and Validation*, 2019.
- [3] M. Harman and P. O’Hearn, “From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis,” in *SCAM 2018: Proceedings of the 18th International Working Conference on Source Code Analysis and Manipulation*, 2018.
- [4] A. Labuschagne, L. Inozemtseva, and R. Holmes, “Measuring the cost of regression testing in practice: A study of Java projects using continuous integration,” in *ESEC/FSE 2017: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017.
- [5] J. Micco, “The state of continuous integration testing at Google,” in *ICST 2017: 10th International Conference on Software Testing, Verification and Validation*, 2017.
- [6] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, and A. Memon, “Modeling and ranking flaky tests at Apple,” in *ICSE SEIP 2020: Proceedings of the 42nd International Conference on Software Engineering, Software Engineering in Practice Track*, 2020.
- [7] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, “Understanding flaky tests: The developer’s perspective,” in *ESEC/FSE 2019: Proceedings of the 2019 13th Joint Meeting on Foundations of Software Engineering*, 2019.
- [8] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, “A survey of flaky tests,” *TOSEM*, 2021.
- [9] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, “Taming Google-scale continuous testing,” in *ICSE SEIP 2017: Proceedings of the 39th International Conference on Software Engineering, Software Engineering in Practice Track*, 2017.
- [10] C. Ziftci and J. Reardon, “Who broke the build?: Automatically identifying changes that induce test failures in continuous integration at Google scale,” in *ICSE SEIP 2017: Proceedings of the 39th International Conference on Software Engineering, Software Engineering in Practice Track*, 2017.
- [11] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, “Root causing flaky tests in a large-scale industrial setting,” in *ISSTA 2019: Proceedings of the 2019 International Symposium on Software Testing and Analysis*, 2019.
- [12] W. Lam, K. Muşlu, H. Sajani, and S. Thummalapenta, “A study on the lifecycle of flaky tests,” in *ICSE 2020: Proceedings of the 42nd International Conference on Software Engineering*, 2020.
- [13] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, “Empirically revisiting the test independence assumption,” in *ISSTA 2014: Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014.
- [14] M. Gruber, S. Lukaszcyk, F. Kroiß, and G. Fraser, “An empirical study of flaky tests in Python,” in *ICST 2021: 14th International Conference on Software Testing, Verification and Validation*, 2021.
- [15] R. Wang, Y. Chen, and W. Lam, “iPFlakies: A framework for detecting and fixing Python order-dependent flaky tests,” in *ICSE Demo 2022: Proceedings of the 44th International Conference on Software Engineering, Demonstrations Track*, 2022.
- [16] C. Li and A. Shi, “Evolution-aware detection of order-dependent flaky tests,” in *ISSTA 2022: Proceedings of the 2022 International Symposium on Software Testing and Analysis*, 2022.
- [17] J. Bell, G. Kaiser, E. Melski, and M. Dattatreya, “Efficient dependency detection for safe Java test acceleration,” in *ESEC/FSE 2015: The 10th joint meeting of the European Software Engineering Conference (ESEC and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE))*, 2015.
- [18] A. Gambi, J. Bell, and A. Zeller, “Practical test dependency detection,” in *ICST 2018: 11th International Conference on Software Testing, Verification and Validation*, 2018.
- [19] A. Wei, P. Yi, T. Xie, D. Marinov, and W. Lam, “Probabilistic and systematic coverage of consecutive test-method pairs for detecting order-dependent flaky tests,” in *TACAS 2021: Tools and Algorithms for the Construction and Analysis of Systems*, 2021.
- [20] C. Li, M. M. Khosravi, W. Lam, and A. Shi, “Systematically producing test-orders to detect order-dependent flaky tests,” in *ISSTA 2023: Proceedings of the 2023 International Symposium on Software Testing and Analysis*, 2023.
- [21] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, “iFixFlakies: A framework for automatically fixing order-dependent flaky tests,” in *ESEC/FSE 2019: Proceedings of the 2019 13th Joint Meeting on Foundations of Software Engineering*, 2019.
- [22] W. Lam, S. Winter, A. Wei, T. Xie, D. Marinov, and J. Bell, “A large-scale longitudinal study of flaky tests,” in *OOPSLA 2020: Object-Oriented Programming Systems, Languages, and Applications*, 2020.
- [23] J. Bell, “Detecting, isolating, and enforcing dependencies among and within test cases,” in *FSE DS 2014: Proceedings of the ACM SIGSOFT 22nd Symposium on the Foundations of Software Engineering, Doctoral Symposium*, 2014.
- [24] F. Eder and S. Winter, “Efficient detection of test interference in c projects,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)*, 2024.
- [25] “Website,” 2025, <https://anonymous.4open.science/t/TuscanE-4B9C>.
- [26] W. Lam, S. Winter, A. Astorga, V. Stodden, and D. Marinov, “Understanding reproducibility and characteristics of flaky tests through test reruns in Java projects,” in *ISSRE 2020: 31st International Symposium on Software Reliability Engineering*, 2020.
- [27] “Dubbo,” 2025, <https://github.com/apache/dubbo>.
- [28] S. W. Golomb and H. Taylor, “Tuscan squares – A new family of combinatorial designs,” vol. 20, no. B, 1985.
- [29] “JUnit,” 2025, <https://junit.org>.
- [30] “Maven Surefire plugin,” 2025, <https://maven.apache.org/surefire/maven-surefire-plugin>.
- [31] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, “Predictive test selection,” in *ICSE SEIP 2019: Proceedings of the 41st International Conference on Software Engineering, Software Engineering in Practice Track*, 2019.
- [32] A. Gyori, A. Shi, F. Hariri, and D. Marinov, “Reliable testing: Detecting state-polluting tests to prevent test dependency,” in *ISSTA 2015: Proceedings of the 2015 International Symposium on Software Testing and Analysis*, 2015.
- [33] S. Rahman, B. N. Chanumolu, S. Rafi, A. Shi, and W. Lam, “Ranking Relevant Tests for Order-Dependent Flaky Tests,” in *ICSE 2025: Proceedings of the 47th International Conference on Software Engineering*, 2025.
- [34] W. Lam, A. Shi, R. Oei, S. Zhang, M. D. Ernst, and T. Xie, “Dependent-test-aware regression testing techniques,” in *ISSTA 2020: Proceedings of the 2020 International Symposium on Software Testing and Analysis*, 2020.
- [35] R. Kuhn, R. Kacker, Y. Lei, and J. Hunter, “Combinatorial software testing,” *Computer*, vol. 42, no. 8, pp. 94–96, 2009.
- [36] E. Baranov, S. Chakraborty, A. Legay, K. S. Meel, and N. Vinodchandran, “A scalable t-wise coverage estimator,” in *ICSE 2022: Proceedings of the 44th International Conference on Software Engineering*, 2022.
- [37] C. Luo, Q. Zhao, S. Cai, H. Zhang, and C. Hu, “SamplingCA: effective and efficient sampling-based pairwise testing for highly configurable software systems,” in *ESEC/FSE 2022: Proceedings of the 2022 16th Joint Meeting on Foundations of Software Engineering*, 2022, pp. 1185–1197.